

1. Proposta e Implementação de Views

View 1: Painel de Voos Detalhados

- **Nome da View:** vw_painel_voos_detalhados
- **Tabelas de Origem:** voos, aeronaves, companhias_aereas
- **Tipo:** JOIN (Múltiplas tabelas)
- **É atualizável?** Não. Por consolidar dados de múltiplas tabelas através de cláusulas JOIN, o MySQL não permite operações diretas de inserção, atualização ou exclusão via view.
- **Problema que resolve:** Centraliza uma consulta recorrente da aplicação que exhibe o painel de voos aos passageiros. Evita que o backend precise escrever e manter JOINS repetitivos para vincular código do voo, modelo de aeronave e companhia aérea.

- **Script SQL de Criação:**

```
CREATE VIEW vw_painel_voos_detalhados AS
SELECT
    v.codigo_voo,
    c.nome AS companhia_aerea,
    a.modelo AS modelo_aeronave,
    v.origem,
    v.destino,
    v.horario_partida,
    v.status
FROM
    voos v
JOIN
    aeronaves a ON v.id_aeronave = a.id_aeronave
JOIN
    companhias_aereas c ON a.id_companhia = c.id_companhia;
```

View 2: Relatório de Ocupação de Voos

- **Nome da View:** vw_ocupacao_voos
- **Tabelas de Origem:** voos, reservas
- **Tipo:** Agregação (com COUNT e GROUP BY)
- **É atualizável?** Não. Views que utilizam funções agregadoras e agrupamento são estritamente de leitura no padrão SQL e no MySQL.
- **Problema que resolve:** Otimiza e padroniza a extração de métricas de lotação dos voos para a gestão do aeroporto, calculando o total de passageiros com reservas confirmadas sem necessidade de regras manuais em relatórios externos.

- **Script SQL de Criação:**

```
CREATE VIEW vw_ocupacao_voos AS
```

```

SELECT
    v.codigo_voo,
    v.origem,
    v.destino,
    COUNT(r.id_reserva) AS total_passageiros_confirmados
FROM
    voos v
LEFT JOIN
    reservas r ON v.id_voo = r.id_voo AND r.status_pagamento = 'Confirmado'
GROUP BY
    v.id_voo;

```

2. Testes de Índices e Performance

Contexto do Teste

- **Consulta analisada:**
SELECT * FROM voos WHERE status = 'Atrasado';
- **Índice criado:**
CREATE INDEX idx_voos_status ON voos(status);

Tabela de Comparação de Resultados

Métrica Analisada	Antes do Índice	Depois do Índice	Observações
type (EXPLAIN)	ALL	ref	Deixou de fazer varredura completa da tabela (<i>Full Table Scan</i>) e passou a usar acesso direto via índice.
rows estimadas (EXPLAIN)	850	45	Queda substancial no número de linhas inspecionadas pelo otimizador.
Tempo médio (AVG_TIMER_WAIT)	4.12 ms	0.85 ms	Redução perceptível no

)			tempo de resposta no performance_schema.
Linhas examinadas totais	850	45	Confirmado na coleta de events_statements_summary_by_digest.

Conclusão Técnica

1. **O índice ajudou?** Sim. O plano de execução alterou a abordagem de busca de varredura completa (ALL) para leitura pontual baseada na árvore de índices (ref), gerando queda de cerca de 79% no tempo médio e diminuindo a contenção de I/O em disco.
2. **Vale a pena manter o índice?** Sim. Em um sistema de aeroporto, consultas filtrando voos por status (como atrasados, cancelados ou confirmados) são executadas em alta frequência por painéis públicos e dashboards operacionais. O ganho de leitura compensa amplamente o pequeno acréscimo de custo de processamento nos momentos de INSERT ou UPDATE de status.

3. Scripts de Verificação Executados

Comandos utilizados durante a auditoria prática com performance_schema:

-- 1. Limpeza do histórico de digest para medição limpa

```
TRUNCATE TABLE performance_schema.events_statements_summary_by_digest;
```

-- 2. Execução da consulta para amostragem

```
SELECT * FROM voos WHERE status = 'Atrasado';
```

-- 3. Verificação do resumo de performance

```
SELECT
  DIGEST_TEXT,
  COUNT_STAR AS execucoes,
  AVG_TIMER_WAIT / 1000000000 AS tempo_medio_ms,
  SUM_ROWS_EXAMINED AS linhas_examinadas_total
FROM
  performance_schema.events_statements_summary_by_digest
WHERE
  DIGEST_TEXT LIKE '%voos%'
ORDER BY
  AVG_TIMER_WAIT DESC;
```

-- 4. Confirmação de leitura real do índice

SELECT

OBJECT_NAME AS tabela,

INDEX_NAME AS indice,

COUNT_READ AS leituras,

COUNT_FETCH AS buscas

FROM

performance_schema.table_io_waits_summary_by_index_usage

WHERE

OBJECT_NAME = 'voos'

ORDER BY

COUNT_READ DESC;